

LINEAR ALGEBRA

for Machine Learning, Deep Learning, AI & Data Science

Beginner-Friendly • Intuition-Driven • 60+ Pages

Complete coverage of every major topic with Python examples,
step-by-step worked examples, and real ML/AI use-cases

Author: Your Name | Date: 2025 | Version 1.0

Designed for students, data scientists, and ML practitioners

Table of Contents

Table of Contents	2
Introduction	6
What is Linear Algebra?	6
Why Linear Algebra is Essential for Machine Learning and AI.....	6
How to Use This Book	7
Chapter 1: Data Representation	8
1.1 Scalars — The Building Block	8
What is a Scalar?	8
Scalars in Machine Learning.....	8
Python Example	8
1.2 Vectors — Ordered Lists of Numbers	9
What is a Vector?	9
Geometric Interpretation.....	9
Vector Operations.....	9
Dot Product	10
Python Code: Vectors.....	10
Where Vectors Are Used in ML/AI	11
1.3 Matrices — Tables of Numbers	11
What is a Matrix?	11
Matrix Addition and Subtraction	12
Matrix Transposition	12
Matrix Multiplication — The Most Important Operation	13
Python Code: Matrices.....	13
1.4 Tensors — The Generalized Array	14
What is a Tensor?.....	14
Python Code: Tensors	15
Chapter 2: Norms and Distance Metrics.....	17
2.1 Vector Norms — Measuring Length	17
What is a Norm?	17
Python Code: Norms	18
2.2 Cosine Similarity — Measuring Direction	18
What is Cosine Similarity?.....	18
Chapter 3: Linear Transformations and Systems	20

3.1 What is a Linear Transformation?	20
Common Geometric Transformations	20
Python Code: Transformations	20
3.2 The Identity and Inverse Matrices	21
Identity Matrix	21
Inverse Matrix	21
Determinant	22
3.3 Solving Linear Systems: $Ax = b$	22
What is a System of Linear Equations?	22
Gaussian Elimination — Step by Step	23
LU and QR Decomposition	24
Chapter 4: Vector Spaces, Span, and Linear Independence	26
4.1 Linear Combinations	26
What is a Linear Combination?	26
4.2 Span	26
What is the Span of Vectors?	26
4.3 Linear Independence	27
What Does 'Linearly Independent' Mean?	27
4.4 Basis and Dimension	27
What is a Basis?	27
4.5 Rank of a Matrix	28
What is Matrix Rank?	28
Chapter 5: Eigenvalues and Eigenvectors	30
5.1 The Big Idea	30
What are Eigenvectors and Eigenvalues?	30
Step-by-Step: Finding Eigenvalues	30
Geometric Meaning	31
Python Code: Eigenvalues and Eigenvectors	31
5.2 Eigendecomposition	31
What is Eigendecomposition?	31
Chapter 6: PCA and Dimensionality Reduction	34
6.1 The Problem: Too Many Dimensions	34
6.2 PCA Step by Step	34
The PCA Algorithm	34
Full Python Implementation	34

Chapter 7: Singular Value Decomposition (SVD)	37
7.1 What is SVD?	37
7.2 Low-Rank Approximation — Compression with SVD.....	37
Python: SVD and Image Compression.....	38
SVD vs PCA	38
Chapter 8: Orthogonality and Projections	40
8.1 Orthogonal and Orthonormal Vectors.....	40
Orthogonal Vectors.....	40
Orthonormal Vectors	40
Orthogonal Matrices	40
8.2 Projections	40
Projecting a Vector onto Another Vector	41
Projecting onto a Subspace.....	41
Chapter 9: Linear Algebra in Machine Learning.....	43
9.1 Linear Regression	43
9.2 Neural Networks — Layers as Matrix Multiplications	43
9.3 Support Vector Machines (SVM).....	44
9.4 Clustering and k-Means	45
9.5 Recommendation Systems.....	45
Chapter 10: Advanced Topics	47
10.1 The Null Space and Column Space.....	47
Column Space.....	47
Null Space (Kernel).....	47
10.2 Gradient Descent and Linear Algebra	47
10.3 Attention Mechanism and Transformers.....	48
10.4 Tensor Operations in Deep Learning	49
Chapter 11: Summary and Further Reading	51
11.1 Core Ideas Recap.....	51
11.2 How These Concepts Connect	51
11.3 Recommended Next Steps.....	52
Mathematics	52
Books.....	52
Online Courses	52
Practice Projects.....	52
11.4 Final Mini Quiz — Everything.....	52

Introduction

What is Linear Algebra?

Linear algebra is the branch of mathematics that deals with vectors, matrices, and linear transformations. If you have ever worked with a spreadsheet of numbers, trained a machine learning model, or even watched a video on a streaming platform that recommended what to watch next — you have benefited from linear algebra, whether you knew it or not.

Think of linear algebra as the language of data. Just as grammar rules help us form sentences in English, linear algebra provides the rules for manipulating and understanding data arranged in tables and arrays. It is the foundation beneath almost every powerful algorithm in modern artificial intelligence.

A Simple Way to Think About It

Linear algebra is about TRANSFORMING data. You start with some numbers, apply mathematical rules, and produce new, useful numbers.

Example: You have 1,000 customers with 50 features each. Linear algebra lets you ask: 'Which features are most important?' and 'How can I compress these 50 features into just 2 for easy visualization?'

Why Linear Algebra is Essential for Machine Learning and AI

Every single machine learning algorithm relies on linear algebra at its core. Here is why:

- Data is stored as matrices. A dataset with 1,000 rows and 20 columns is literally a $1,000 \times 20$ matrix.
- Neural networks multiply matrices billions of times during training.
- PCA (Principal Component Analysis) uses eigenvalues to find patterns in data.
- Image recognition treats every image as a matrix of pixel values.
- Recommendation systems (Netflix, Spotify) use matrix factorization to suggest content.
- Natural Language Processing (NLP) represents words as vectors in high-dimensional space.

ML Concept	Linear Algebra Used
Linear Regression	Matrix equations, dot products, least squares
Neural Networks	Matrix multiplication, weight matrices, gradients

PCA	Eigenvalues, eigenvectors, covariance matrix
Image Processing	2D/3D matrices, convolutions, tensor operations
Word Embeddings	Vectors, cosine similarity, SVD
Recommendation	Matrix factorization, SVD, low-rank approximation
SVM	Dot products, projections, hyperplane geometry
Clustering (k-means)	Euclidean distance, norms, centroids

How to Use This Book

This book is structured so that each topic follows the same pattern: intuition first, formal math second, then real-world application, Python code, and finally practice problems. You do not need any prior knowledge of linear algebra. A basic familiarity with high school algebra and Python is all you need.

- Read each section sequentially — topics build on each other.
- Do not skip the analogies — they help the math stick.
- Run every Python code snippet in a Jupyter notebook.
- Attempt the mini quiz before reading the answers.

Key Insight

Mathematics is like weightlifting. The first time you pick up a heavy weight, it feels impossible. But with repeated practice, it becomes easy. The same goes for linear algebra — the first time is hard, but after a few examples, it all clicks.

Chapter 1: Data Representation

1.1 Scalars — The Building Block

What is a Scalar?

A scalar is simply a single number. That is all. It has magnitude (size) but no direction. When you say 'the temperature today is 25 degrees' or 'I have 3 apples', you are talking about scalars.

Formal Definition

A scalar is a single real number. In mathematics, we usually denote scalars with lowercase italic letters:

$$x = 5 \quad y = -3.14 \quad \lambda = 0.001$$

Scalars in Machine Learning

- Learning rate ($\alpha = 0.001$) — controls how fast a neural network learns
- Loss value ($L = 0.42$) — measures how wrong a model's predictions are
- Regularization parameter ($\lambda = 0.01$) — controls model complexity
- Probability output ($p = 0.87$) — output of a classifier

Real-World Analogy

A scalar is like the thermostat reading in your house. It is ONE number that tells you something specific — the temperature. By itself, it has no direction; it just has a value.

Python Example

```
import numpy as np

# Defining scalars
learning_rate = 0.001
loss_value    = 0.42
temperature   = 25.0

# Scalar arithmetic
result = learning_rate * loss_value
print(f'Scaled loss: {result}')    # 0.00042

# In NumPy, a scalar is a 0-dimensional array
x = np.float64(5.0)
print(x.shape)    # ()
```

```
print(x.ndim)    # 0
```

1.2 Vectors — Ordered Lists of Numbers

What is a Vector?

A vector is an ordered list of numbers. Where a scalar has just one value, a vector has multiple values arranged in a sequence. Those values are called the components or elements of the vector.

You can think of a vector in two ways. First, as a list of measurements: [height=170cm, weight=70kg, age=25]. Second, as an arrow in space pointing from the origin (0,0,0) to a specific point.

Column vector: $v = [v_1, v_2, v_3, \dots, v_n]^T$

Example: $v = [3, 4]^T$ (a 2D vector pointing to the point (3,4))

Geometric Interpretation

Imagine you are standing at the origin (the centre of a graph). A vector [3, 4] is an instruction: 'Walk 3 steps to the right, then 4 steps up.' The arrow from where you started to where you ended is the vector.

Vectors in ML: What do they represent?

A single customer record: [age=25, salary=50000, purchases=12]

A single image (flattened): [pixel₁, pixel₂, ..., pixel₇₈₄] for a 28×28 MNIST image

A word embedding: a 300-dimensional vector representing the word 'king'

A neural network weight vector: all weights connecting two layers

Vector Operations

Vector Addition

Adding two vectors means adding their corresponding elements. The vectors must have the same number of elements (same dimension).

$$a + b = [a_1 + b_1, a_2 + b_2, \dots, a_n + b_n]$$

Example: $a = [1, 2, 3]$ and $b = [4, 5, 6]$

$$a + b = [1+4, 2+5, 3+6] = [5, 7, 9]$$

Real-World Analogy

Imagine two friends each give you directions from your home. Friend A says: go 3km east, 2km north. Friend B says: go 1km east, 4km north. If you follow BOTH sets of directions, you end up at $[3+1, 2+4] = [4, 6]$ — 4km east and 6km north. That is vector addition.

Scalar Multiplication

Multiplying a vector by a scalar stretches or shrinks it. Each element gets multiplied by the scalar.

$$c \cdot v = [c \cdot v_1, c \cdot v_2, \dots, c \cdot v_n]$$

Example: $3 \times [2, 5] = [6, 15]$

Dot Product

The dot product is one of the MOST important operations in machine learning. It takes two vectors of the same length and produces a single number (a scalar).

$$a \cdot b = a_1b_1 + a_2b_2 + \dots + a_nb_n = \sum(a_i \times b_i)$$

Example: $a = [1, 2, 3]$, $b = [4, 5, 6]$

$$a \cdot b = (1 \times 4) + (2 \times 5) + (3 \times 6) = 4 + 10 + 18 = 32$$

Why Does the Dot Product Matter?

The dot product measures how much two vectors point in the same direction.

- If $a \cdot b$ is LARGE and POSITIVE $\rightarrow$ the vectors point mostly in the same direction
- If $a \cdot b = 0 \rightarrow$ the vectors are perpendicular (orthogonal)
- If $a \cdot b$ is NEGATIVE $\rightarrow$ the vectors point in opposite directions

In neural networks: the output of a neuron is computed as the dot product of inputs and weights, plus a bias.

Python Code: Vectors

```

import numpy as np

# Define vectors
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Vector addition
print('a + b =', a + b)          # [5 7 9]

# Scalar multiplication
print('3 * a =', 3 * a)          # [3 6 9]

# Dot product (three equivalent ways)
print('Dot product:', np.dot(a, b)) # 32
print('Dot product:', a @ b)        # 32
print('Dot product:', sum(a * b))   # 32

# Vector norm (length)
print('Length of a:', np.linalg.norm(a)) # 3.7417

# Unit vector (direction only, length = 1)
unit_a = a / np.linalg.norm(a)
print('Unit vector:', unit_a)
print('Its length:', np.linalg.norm(unit_a)) # 1.0

```

Where Vectors Are Used in ML/AI

- Linear Regression: input features are a vector $x = [x_1, x_2, \dots, x_n]$
- Neural Networks: activations of a layer are a vector; weights are vectors
- Word2Vec / GloVe: each word is represented as a dense vector (word embedding)
- k-Nearest Neighbours: computes distances between feature vectors
- Cosine Similarity in NLP: measures angle between document vectors

Mini Quiz

Q1: What is the dot product of [2, 3] and [4, 1]?

Answer: $2 \times 4 + 3 \times 1 = 8 + 3 = 11$

Q2: If a vector $a = [3, 4]$, what is its length (L2 norm)?

Answer: $\sqrt{3^2 + 4^2} = \sqrt{9+16} = \sqrt{25} = 5$

Q3: What does a dot product of 0 tell you about two vectors?

Answer: The vectors are perpendicular (orthogonal) to each other — they are completely unrelated in direction.

1.3 Matrices — Tables of Numbers

What is a Matrix?

A matrix is a 2-dimensional grid of numbers arranged in rows and columns. It is like a spreadsheet or a table. We describe the size of a matrix by its dimensions: $m \times n$, meaning m rows and n columns.

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix} \quad (\text{a } 2 \times 3 \text{ matrix: 2 rows, 3 columns})$$

$$A = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$$

The element in row i and column j is written as A_{ij} . So $A_{12} = 3$ (row 1, column 2) and $A_{23} = 6$ (row 2, column 3).

A Dataset IS a Matrix

Think of a dataset with 1,000 customer records, each with 5 features: age, salary, city_code, num_purchases, churn_risk.

This is a 1000×5 matrix where:

- Each ROW is one customer (one data point / sample)
- Each COLUMN is one feature

Every machine learning algorithm you will ever train operates on this kind of matrix.

Matrix Addition and Subtraction

You can add or subtract two matrices only if they have the same dimensions. Just add/subtract corresponding elements.

$$(A + B)_{ij} = A_{ij} + B_{ij}$$

Matrix Transposition

The transpose of a matrix flips it: rows become columns and columns become rows. If A is $m \times n$, then A^T is $n \times m$.

$$(A^T)_{ij} = A_{ji}$$

Example:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \rightarrow A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Matrix Multiplication — The Most Important Operation

Matrix multiplication is not just element-wise. To multiply matrix A (m×n) by matrix B (n×p), the number of COLUMNS in A must equal the number of ROWS in B. The result is an m×p matrix.

$$(AB)_{ij} = \sum_k A_{ik} \cdot B_{kj} \quad (\text{dot product of row } i \text{ of } A \text{ with column } j \text{ of } B)$$

Step-by-step example:

$$\begin{matrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} & \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} & \begin{bmatrix} (1 \times 5 + 2 \times 7) & (1 \times 6 + 2 \times 8) \\ (3 \times 5 + 4 \times 7) & (3 \times 6 + 4 \times 8) \end{bmatrix} \\ A & B & AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix} \end{matrix}$$

Neural Network Forward Pass = Matrix Multiplication

A neural network layer computes: $\text{output} = \text{activation}(W \times x + b)$

Where:

- W is the weight matrix
- x is the input vector
- b is the bias vector

If W is (100×50) and x is (50×1), the output has shape (100×1).

Matrix multiplication allows ONE operation to compute outputs for ALL neurons simultaneously.

Python Code: Matrices

```
import numpy as np

# Define matrices
A = np.array([[1, 2, 3],
              [4, 5, 6]])

B = np.array([[7, 8],
              [9, 10],
              [11, 12]])
```

```

print('Shape of A:', A.shape)    # (2, 3)
print('Shape of B:', B.shape)    # (3, 2)

# Matrix multiplication (2x3) @ (3x2) = (2x2)
C = A @ B
print('A @ B =\n', C)
# [[ 58  64]
#  [139 154]]

# Transpose
print('AT =\n', A.T)

# Element-wise operations
X = np.array([[1,2],[3,4]])
Y = np.array([[5,6],[7,8]])
print('Element-wise multiply:', X * Y)

# Simulating a neural network layer
W = np.random.randn(100, 50) # Weight matrix
x = np.random.randn(50, 1)   # Input vector
b = np.random.randn(100, 1)   # Bias
output = W @ x + b
print('Output shape:', output.shape) # (100, 1)

```

Mini Quiz

Q1: A matrix A is (3×4) and matrix B is (4×2). What is the shape of A@B?

Answer: 3×2 (rows of A × columns of B)

Q2: Can you multiply a (3×4) matrix by a (3×4) matrix directly?

Answer: No! Columns of first (4) must equal rows of second (3). Here 4 ≠ 3, so it is impossible.

Q3: What does the transpose operation do geometrically?

Answer: It reflects the matrix along its main diagonal, swapping rows and columns.

1.4 Tensors — The Generalized Array

What is a Tensor?

A tensor is simply a multi-dimensional array. You already know scalars (0D), vectors (1D), and matrices (2D). A tensor generalizes this to any number of dimensions.

Tensor Rank	Shape Example	Real-World Example
0 (Scalar)	()	A single loss value: 0.42
1 (Vector)	(n,)	Feature vector of one customer: (50,)
2 (Matrix)	(m, n)	Dataset: (1000 rows, 20 features)
3 (3D Tensor)	(d, h, w)	Grayscale image batch: (32 images, 28 pixels, 28 pixels)

4 (4D Tensor)	(b, c, h, w)	Color image batch: (32 images, 3 channels, 224, 224)
5 (5D Tensor)	(b, t, c, h, w)	Video batch: (8 videos, 30 frames, 3 ch, 224, 224)

RGB Images as Tensors

A single colour photograph (224×224 pixels with R, G, B channels) is a 3D tensor of shape (3, 224, 224).

When you process a BATCH of 32 such images in a neural network, the tensor shape becomes (32, 3, 224, 224).

Deep learning frameworks like PyTorch and TensorFlow are built entirely around tensor operations.

Python Code: Tensors

```
import numpy as np

# 0D tensor (scalar)
scalar = np.float32(3.14)
print('0D shape:', scalar.shape)    # ()

# 1D tensor (vector)
vec = np.array([1, 2, 3, 4])
print('1D shape:', vec.shape)       # (4,)

# 2D tensor (matrix)
mat = np.random.randn(5, 3)
print('2D shape:', mat.shape)       # (5, 3)

# 3D tensor (batch of grayscale images: 8 images, 28×28 pixels)
images_gray = np.random.rand(8, 28, 28)
print('3D shape:', images_gray.shape) # (8, 28, 28)

# 4D tensor (batch of RGB images: 32 images, 3 channels, 64×64)
images_rgb = np.random.rand(32, 3, 64, 64)
print('4D shape:', images_rgb.shape)  # (32, 3, 64, 64)

# Basic tensor ops
a = np.random.rand(3, 4, 5)
b = np.random.rand(3, 4, 5)
print('Sum shape:', (a + b).shape)    # (3, 4, 5)
print('Element product:', (a * b).shape) # (3, 4, 5)
```

Mini Quiz

Q1: What is the shape of a batch of 64 colour images, each 128×128 pixels?

Answer: (64, 3, 128, 128) — batch=64, channels=3 (RGB), height=128, width=128

Q2: Is every matrix a tensor?

Answer: Yes! A matrix is a rank-2 tensor. Every matrix is a tensor, but not every tensor is a matrix.

Chapter 2: Norms and Distance Metrics

2.1 Vector Norms — Measuring Length

What is a Norm?

A norm is a function that takes a vector and returns a non-negative number representing its 'length' or 'size'. Different norms measure length in different ways. The most common norms in ML are L1, L2, and L^∞ .

L1 Norm (Manhattan): $\|v\|_1 = |v_1| + |v_2| + \dots + |v_n|$

L2 Norm (Euclidean): $\|v\|_2 = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$

L^∞ Norm (Chebyshev): $\|v\|_\infty = \max(|v_1|, |v_2|, \dots, |v_n|)$

General L_p Norm: $\|v\|_p = (|v_1|^p + |v_2|^p + \dots + |v_n|^p)^{1/p}$

Example: For $v = [3, -4, 0]$:

- L1 norm: $|3| + |-4| + |0| = 3 + 4 + 0 = 7$
- L2 norm: $\sqrt{9 + 16 + 0} = \sqrt{25} = 5$
- L^∞ norm: $\max(3, 4, 0) = 4$

Real-World Analogy

Imagine navigating New York City. The L1 norm is like taxi distance — you can only travel along the street grid (no diagonals). The L2 norm is like the straight-line (as-the-crow-flies) distance — the shortest possible path if there were no buildings.

Norms in Machine Learning

L1 Norm (Lasso Regularization): Adds sum of absolute weights to loss. Creates SPARSE models — forces some weights to exactly zero. Great for feature selection.

L2 Norm (Ridge Regularization): Adds sum of squared weights to loss. Shrinks all weights gradually. Prevents overfitting. Also used to normalize feature vectors.

Frobenius Norm: The L2 norm for matrices — square root of sum of squared elements. Used in matrix approximation and PCA.

Python Code: Norms

```
import numpy as np

v = np.array([3.0, -4.0, 0.0])

l1 = np.linalg.norm(v, ord=1)    # 7.0
l2 = np.linalg.norm(v, ord=2)    # 5.0
linf = np.linalg.norm(v, ord=np.inf) # 4.0

print(f'L1 norm: {l1}')
print(f'L2 norm: {l2}')
print(f'L-inf norm: {linf}')

# Frobenius norm of a matrix
M = np.array([[1,2],[3,4]])
frob = np.linalg.norm(M, 'fro')
print(f'Frobenius: {frob:.4f}') # 5.4772

# Normalizing a vector to unit length
unit_v = v / np.linalg.norm(v)
print(f'Unit vector: {unit_v}') # [ 0.6 -0.8  0. ]
print(f'It's L2 norm: {np.linalg.norm(unit_v)}') # 1.0
```

2.2 Cosine Similarity — Measuring Direction

What is Cosine Similarity?

Cosine similarity measures the angle between two vectors, ignoring their magnitudes. Two vectors that point in the same direction have cosine similarity of +1. Perpendicular vectors score 0. Opposite vectors score -1.

$$\cos(\theta) = \frac{a \cdot b}{||a|| \times ||b||}$$

Example: $a = [1, 1]$ and $b = [2, 2]$

$$a \cdot b = 1 \times 2 + 1 \times 2 = 4$$

$$||a|| = \sqrt{2}, \quad ||b|| = \sqrt{8} = 2\sqrt{2}$$

$$\cos(\theta) = 4 / (\sqrt{2} \times 2\sqrt{2}) = 4/4 = 1.0 \rightarrow \text{same direction}$$

Cosine Similarity in NLP

In Natural Language Processing, each document or word is represented as a vector. Cosine similarity tells us how semantically similar two texts are.

Example: 'I love football' and 'I enjoy soccer' would have HIGH cosine similarity even though the exact words differ.

Search engines, recommendation systems, and chatbots all use cosine similarity constantly.

```
import numpy as np
from numpy.linalg import norm

def cosine_similarity(a, b):
    return np.dot(a, b) / (norm(a) * norm(b))

# Word embeddings example
king = np.array([0.9, 0.3, 0.7])
queen = np.array([0.8, 0.4, 0.6])
banana = np.array([0.1, 0.9, 0.2])

print('king vs queen:', cosine_similarity(king, queen)) # ~0.997
print('king vs banana:', cosine_similarity(king, banana)) # ~0.481
```

Mini Quiz

Q1: Which norm produces sparse models in regularization?

Answer: L1 norm (Lasso regularization) drives some weights to exactly zero, creating sparse models.

Q2: If cosine similarity between two documents is 0, what does that tell us?

Answer: The documents are orthogonal — they share no common vocabulary or meaning. They are completely unrelated.

Q3: A model has weights $w = [3, 4]$. What is its L2 norm?

Answer: $\|w\|_2 = \sqrt{9 + 16} = 5$

Chapter 3: Linear Transformations and Systems

3.1 What is a Linear Transformation?

A linear transformation is a function that takes a vector as input and outputs another vector, using a matrix multiplication. It transforms the 'space' in predictable, structured ways — scaling, rotating, reflecting, or shearing — while preserving the origin and straight lines.

$$y = Ax \quad (\text{matrix } A \text{ transforms vector } x \text{ into vector } y)$$

For a transformation T to be 'linear', it must satisfy two rules:

- Additivity: $T(u + v) = T(u) + T(v)$
- Scalar multiplication: $T(c \cdot v) = c \cdot T(v)$

Common Geometric Transformations

Transformation	What it does	Matrix Form
Scaling	Stretches or shrinks space	$[[sx, 0], [0, sy]]$
Rotation (θ)	Rotates space by angle θ	$[[\cos\theta, -\sin\theta], [\sin\theta, \cos\theta]]$
Reflection	Flips across an axis	$[[1, 0], [0, -1]]$ for x-axis
Projection	Projects onto a subspace	$A \cdot A^T (A^T A)^{-1} A^T$
Shear	Slants along one axis	$[[1, k], [0, 1]]$

Real-World Analogy

Think of a matrix as a magic rubber sheet. When you multiply a matrix by a vector, it is like stretching, rotating, or squishing that rubber sheet. The vector (a point on the sheet) moves to wherever the stretched sheet places it.

Python Code: Transformations

```
import numpy as np
import math

# Scale by 2 in x-direction, 3 in y-direction
scale = np.array([[2, 0],
                  [0, 3]])
v = np.array([1, 1])
print('Scaled:', scale @ v)      # [2, 3]
```

```
# Rotate by 90 degrees
theta = math.pi / 2
rotation = np.array([[math.cos(theta), -math.sin(theta)],
                    [math.sin(theta),  math.cos(theta)]])
print('Rotated:', np.round(rotation @ v)) # [-1, 1]

# Compose transformations: first scale, then rotate
combined = rotation @ scale
print('Scale then rotate:', np.round(combined @ v)) # [-3, 2]
```

3.2 The Identity and Inverse Matrices

Identity Matrix

The identity matrix I is the 'do nothing' matrix. Multiplying any matrix or vector by the identity matrix leaves it unchanged — just like multiplying a number by 1.

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

For any matrix A : $A \times I = I \times A = A$

Inverse Matrix

The inverse of a square matrix A is written A^{-1} . It 'undoes' the transformation of A . Think of it as the reverse gear of a transformation.

$$A \times A^{-1} = A^{-1} \times A = I$$

Not all matrices have an inverse. A matrix is invertible only if its determinant is non-zero. A matrix with no inverse is called singular.



Matrix Inverse in ML

The normal equation for linear regression uses the matrix inverse:

$$w = (X^T X)^{-1} X^T y$$

Where X is the feature matrix, y is the target vector, and w gives the optimal weights.

 In practice, gradient descent is used instead because computing inverses is very slow for large matrices ($O(n^3)$ complexity).

Determinant

The determinant is a single number that summarises key properties of a square matrix. It tells you how much the matrix scales areas (in 2D) or volumes (in 3D).

For 2×2 : $\det(\begin{bmatrix} a & b \\ c & d \end{bmatrix}) = ad - bc$

$\det(A) = 0 \rightarrow$ matrix is singular (no inverse, rows are linearly dependent)

$\det(A) \neq 0 \rightarrow$ matrix is invertible

```
import numpy as np

A = np.array([[3, 1],
              [2, 4]])

# Inverse
A_inv = np.linalg.inv(A)
print('A inverse:\n', A_inv)

# Verify: A @ A_inv should be identity
print('A @ A_inv:\n', np.round(A @ A_inv))

# Determinant
det = np.linalg.det(A)
print('det(A):', det)    # 3*4 - 1*2 = 10

# Singular matrix (det = 0)
B = np.array([[1, 2],
              [2, 4]])
print('det(B):', np.linalg.det(B))    # ~0 (singular!)
```

3.3 Solving Linear Systems: $Ax = b$

What is a System of Linear Equations?

A linear system is a set of equations where every equation is a linear combination of the unknowns. For example:

$$2x + 3y = 8$$

$$5x - y = 2$$

In matrix form: $Ax = b$

$$\begin{bmatrix} 2 & 3 \\ 5 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 8 \\ 2 \end{bmatrix}$$

The solution is $x = A^{-1}b$ (if A is invertible). In practice, we use efficient algorithms like Gaussian Elimination rather than computing the full inverse.

Gaussian Elimination — Step by Step

Gaussian elimination is a systematic method for solving linear systems. It uses row operations to transform the matrix into an upper triangular form, then back-substitutes to find the unknowns.

Row Operations (allowed without changing the solution)

1. Swap two rows
2. Multiply a row by a non-zero scalar
3. Add a multiple of one row to another row

Example: Solve the system: $x + 2y = 5$ and $3x + y = 10$

- Write augmented matrix: $[[1, 2, |, 5], [3, 1, |, 10]]$
- $R2 = R2 - 3 \times R1$: $[[1, 2, |, 5], [0, -5, |, -5]]$
- $R2 = R2 / (-5)$: $[[1, 2, |, 5], [0, 1, |, 1]]$
- $R1 = R1 - 2 \times R2$: $[[1, 0, |, 3], [0, 1, |, 1]]$
- Solution: $x = 3, y = 1$

```
import numpy as np

# Define the system Ax = b
A = np.array([[1., 2.],
              [3., 1.]])
b = np.array([5., 10.])

# Solve using numpy
x = np.linalg.solve(A, b)
print('Solution x:', x)    # [3. 1.]

# Verify: A @ x should equal b
print('Verify A@x:', A @ x)  # [5. 10.]

# For overdetermined systems (more equations than unknowns)
# Use least squares
A_over = np.random.randn(100, 3)  # 100 equations, 3 unknowns
```

```
b_over = np.random.randn(100)
x_ls, _, _ = np.linalg.lstsq(A_over, b_over, rcond=None)
print('Least squares solution:', x_ls)
```

LU and QR Decomposition

Rather than computing inverses directly (slow and numerically unstable), we decompose matrices into simpler factors.

LU Decomposition: $A = L \times U$

L = Lower triangular matrix (zeros above diagonal)

U = Upper triangular matrix (zeros below diagonal)

Why use it? Solving $LUx = b$ is much faster than computing A^{-1} .

Used in: Gaussian elimination, solving many right-hand sides efficiently.

QR Decomposition: $A = Q \times R$

Q = Orthogonal matrix ($Q^T = Q^{-1}$, columns are orthonormal)

R = Upper triangular matrix

Why use it? Numerically stable. Q preserves lengths and angles.

Used in: Least squares regression, eigenvalue algorithms.

```
import numpy as np
from scipy import linalg

A = np.array([[2., 1., 0.],
              [1., 3., 1.],
              [0., 1., 2.]])

# LU decomposition
P, L, U = linalg.lu(A)
print('L =\n', np.round(L, 3))
print('U =\n', np.round(U, 3))

# QR decomposition
Q, R = np.linalg.qr(A)
print('Q =\n', np.round(Q, 3))
print('R =\n', np.round(R, 3))
```

Mini Quiz

Q1: When does a matrix NOT have an inverse?

Answer: When its determinant is zero. Such matrices are called singular. Their rows are linearly dependent (one row is a combination of others).

Q2: Why is Gaussian elimination preferred over computing A^{-1} directly?

Answer: It is faster ($O(n^2)$ per solve vs $O(n^3)$ for full inverse), more numerically stable, and works even when A is nearly singular.

Q3: What property makes QR decomposition useful for least squares?

Answer: Q is orthogonal, meaning $Q^T = Q^{-1}$. This makes it very numerically stable and avoids ill-conditioning problems.

Chapter 4: Vector Spaces, Span, and Linear Independence

4.1 Linear Combinations

What is a Linear Combination?

A linear combination of vectors is formed by multiplying each vector by a scalar and adding the results. It is the fundamental building block of everything in linear algebra.

$$c_1v_1 + c_2v_2 + \dots + c_kv_k \quad \text{where } c_1, c_2, \dots, c_k \text{ are scalars}$$

Example: Given $v_1 = [1, 0]$ and $v_2 = [0, 1]$, the vector $[3, -2]$ is a linear combination: $3 \cdot v_1 + (-2) \cdot v_2 = [3, 0] + [0, -2] = [3, -2]$.



Linear Combinations in Neural Networks

Every neuron in a neural network computes a linear combination of its inputs, then applies an activation function:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

The output of the linear layer is literally a linear combination of inputs weighted by the learned weights.

4.2 Span

What is the Span of Vectors?

The span of a set of vectors is all possible vectors you can create through linear combinations of those vectors. Think of it as 'the space you can reach' using those vectors as building blocks.

- Span of $[1, 0]$ alone: all vectors of the form $[c, 0]$ — the x-axis only
- Span of $[1, 0]$ and $[0, 1]$: all 2D vectors $[a, b]$ — the entire 2D plane
- Span of $[1, 0]$ and $[2, 0]$: still just the x-axis! Both vectors point the same way



Real-World Analogy

Imagine you have toy building blocks. The 'span' is all the structures you can build using combinations of those blocks. If all your blocks are the same shape and size, you cannot build very

varied structures — your span is limited. But with blocks of different shapes, you can build almost anything.

4.3 Linear Independence

What Does 'Linearly Independent' Mean?

A set of vectors is linearly independent if no vector in the set can be expressed as a linear combination of the others. In other words, each vector adds something genuinely new — it points in a direction that no combination of the others can reach.

Formally: vectors $v_1, v_2, \dots, v_k$ are linearly independent if the only solution to:

$$c_1v_1 + c_2v_2 + \dots + c_kv_k = 0 \quad \text{is} \quad c_1 = c_2 = \dots = c_k = 0$$

Why This Matters for ML

Linear independence is crucial for feature engineering. If two features in your dataset are linearly dependent (e.g., `weight_in_kg` and `weight_in_pounds`), you are not adding any new information — just noise and computational overhead.

In regression models, linearly dependent features cause multicollinearity, which:

- Makes model weights unstable
- Inflates coefficient standard errors
- Makes interpretation impossible

4.4 Basis and Dimension

What is a Basis?

A basis is a set of linearly independent vectors that span the entire vector space. It is the minimum set of vectors needed to represent every vector in that space. Every vector in the space can be written uniquely as a linear combination of basis vectors.

The standard basis in 2D: $e_1 = [1, 0]$ and $e_2 = [0, 1]$

Every 2D vector $[a, b] = a \cdot e_1 + b \cdot e_2$

The Number of Basis Vectors = Dimension

The dimension of a vector space is the number of vectors in any basis for it.

- 2D plane: 2 basis vectors $\rightarrow$ dimension 2
- 3D space: 3 basis vectors $\rightarrow$ dimension 3
- A dataset with 500 features: lives in 500-dimensional space (at most)

PCA reduces data to a lower-dimensional basis that explains the most variance.

4.5 Rank of a Matrix

What is Matrix Rank?

The rank of a matrix is the number of linearly independent rows (or columns) it has. It tells you the effective dimension of the information in the matrix.

$\text{rank}(A) = \text{number of linearly independent rows} = \text{number of linearly independent columns}$

Full rank matrix: $\text{rank}(A) = \min(m, n) \rightarrow$ all rows/cols are independent

Rank-deficient: $\text{rank}(A) < \min(m, n) \rightarrow$ some rows/cols are redundant

```
import numpy as np

# Full rank matrix
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 10]])
print('Rank of A:', np.linalg.matrix_rank(A)) # 3 (full rank)

# Rank-deficient matrix (row 3 = row 1 + row 2)
B = np.array([[1, 2, 3],
              [4, 5, 6],
              [5, 7, 9]])
print('Rank of B:', np.linalg.matrix_rank(B)) # 2 (rank deficient)

# Checking linear independence
# If det != 0, rows are linearly independent
print('det(A):', np.round(np.linalg.det(A))) # non-zero  $\rightarrow$  independent
print('det(B):', np.round(np.linalg.det(B))) #  $\sim 0 \rightarrow$  dependent
```

Mini Quiz

Q1: Vectors $[1,0]$ and $[0,1]$ are linearly independent. True or False?

Answer: True! Neither can be written as a multiple of the other. Together they form the standard basis for 2D space.

Q2: What is the rank of the matrix $\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$?

Answer: Rank 1. Row 2 is exactly $2 \times$ Row 1, so there is only one linearly independent row.

Q3: Why does multicollinearity hurt regression models?

Answer: When features are linearly dependent, the matrix $X^T X$ becomes nearly singular (not invertible), making weight estimates unstable and unreliable.

Chapter 5: Eigenvalues and Eigenvectors

5.1 The Big Idea

What are Eigenvectors and Eigenvalues?

Most vectors change direction when multiplied by a matrix. But special vectors — called eigenvectors — only get scaled (stretched or shrunk) by the matrix, never rotated. The scaling factor is called the eigenvalue.

$$A \cdot v = \lambda \cdot v \quad \text{where } v \text{ is an eigenvector and } \lambda \text{ (lambda) is the eigenvalue}$$

In words: 'When I multiply matrix A by vector v, the result is just λ times v — the same direction, just a different length.'

Real-World Analogy

Imagine a crowd of people all walking through a revolving door. Most people get spun around (their direction changes). But one person who walks exactly along the rotation axis just gets pushed forward — same direction, just a different speed. That person is like an eigenvector. Their speed multiplier is the eigenvalue.

Step-by-Step: Finding Eigenvalues

To find eigenvalues, we solve the characteristic equation:

$$\det(A - \lambda I) = 0$$

$$\text{Example: } A = \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix}$$

$$\det\left(\begin{bmatrix} 3-\lambda & 1 \\ 0 & 2-\lambda \end{bmatrix}\right) = (3-\lambda)(2-\lambda) - 0 = 0$$

$$\lambda^2 - 5\lambda + 6 = 0$$

$$(\lambda-3)(\lambda-2) = 0$$

$$\lambda_1 = 3, \quad \lambda_2 = 2$$

For each eigenvalue λ , find the eigenvector by solving $(A - \lambda I)v = 0$:

- For $\lambda_1 = 3$: $(A - 3I)v = 0 \rightarrow v_1 = [1, 0]$

- For $\lambda_2 = 2$: $(A - 2I)v = 0 \rightarrow v_2 = [1, -1]$

Geometric Meaning

The eigenvectors are the 'natural axes' of a transformation. The eigenvalues tell you how much the transformation stretches or shrinks along each axis:

- $\lambda > 1$: stretch (expansion)
- $0 < \lambda < 1$: shrink (compression)
- $\lambda = 1$: no change in that direction
- $\lambda < 0$: flip and scale
- $\lambda = 0$: collapse to lower dimension (singular transformation)

Python Code: Eigenvalues and Eigenvectors

```
import numpy as np

A = np.array([[3., 1.],
              [0., 2.]])

eigenvalues, eigenvectors = np.linalg.eig(A)

print('Eigenvalues:', eigenvalues)      # [3. 2.]
print('Eigenvectors:\n', eigenvectors)

# Verify: A @ v = λ @ v for each eigenvector
for i in range(len(eigenvalues)):
    lam = eigenvalues[i]
    vec = eigenvectors[:, i]
    print(f'lambda={lam:.1f}  A@v = {A@vec}  lam*v = {lam*vec}')
```

```
# Symmetric matrix (all eigenvalues are real)
B = np.array([[4., 2.],
              [2., 3.]])
evals, evecs = np.linalg.eigh(B) # eigh for symmetric
print('\nSymmetric eigenvalues:', evals)
```

5.2 Eigendecomposition

What is Eigendecomposition?

If a matrix A has n linearly independent eigenvectors, we can decompose it into three matrices:

$$A = Q \cdot \Lambda \cdot Q^{-1}$$

Q = matrix of eigenvectors (columns)

Λ = diagonal matrix with eigenvalues on the diagonal

For symmetric matrices (very common in ML), Q is orthogonal ($Q^T = Q^{-1}$), so:

$$A = Q \cdot \Lambda \cdot Q^T \quad (\text{symmetric eigendecomposition})$$

Eigendecomposition in Machine Learning

Covariance Matrix: The eigenvectors of the covariance matrix are the principal components (PCA directions). The eigenvalues tell you how much variance each component captures.

Graph Neural Networks: Graph Laplacian eigendecomposition is fundamental to spectral graph theory.

Power Iteration: Google's original PageRank algorithm finds the dominant eigenvector of a link matrix.

```
import numpy as np

# Symmetric matrix (covariance-like)
A = np.array([[4., 2.],
              [2., 3.]])

Q, Lambda = np.linalg.eigh(A) # eigh = symmetric eigen
print('Eigenvalues (diagonal of Λ):', Q)
print('Eigenvectors (columns of V):\n', Lambda)

# Reconstruct: A = V @ diag(Q) @ V.T
V = Lambda
A_reconstructed = V @ np.diag(Q) @ V.T
print('Reconstructed:\n', np.round(A_reconstructed, 6))

# Power to eigenvalues = power to matrix
# A^3 = V @ diag(lambda^3) @ V.T
A_cubed = V @ np.diag(Q**3) @ V.T
print('A^3 via eigen:', np.round(A_cubed))
print('A^3 direct: ', np.round(A @ A @ A))
```

Mini Quiz

Q1: An eigenvector of A satisfies $A \cdot v = 3 \cdot v$. What does this mean geometrically?

Answer: When the transformation A is applied, the vector v only gets scaled by 3 (tripled in length). Its direction does not change.

Q2: What does it mean if an eigenvalue $\lambda = 0$?

Answer: The transformation collapses that direction to zero — information is lost. The matrix is singular (no inverse). The null space is non-trivial.

Q3: Why are eigenvalues of covariance matrices always non-negative?

Answer: Covariance matrices are positive semi-definite by definition. Their eigenvalues represent variance along each principal direction, and variance cannot be negative.

Chapter 6: PCA and Dimensionality Reduction

6.1 The Problem: Too Many Dimensions

Modern datasets often have hundreds or thousands of features. This 'curse of dimensionality' causes problems: models overfit, computation is slow, visualization is impossible, and many features are redundant. PCA solves this by finding the most important directions in the data.

Real-World Analogy

Imagine a dataset where each data point is a shadow of a 3D object. PCA is like finding the best angle to shine a flashlight so that the shadow preserves as much information as possible — compressing 3D into 2D with minimal loss of information.

6.2 PCA Step by Step

The PCA Algorithm

1. Standardize the data: subtract the mean of each feature, divide by standard deviation.
2. Compute the covariance matrix: $C = (1/n) X^T X$ (after centering).
3. Compute eigenvectors and eigenvalues of C .
4. Sort eigenvectors by their eigenvalues in descending order.
5. Select the top k eigenvectors (principal components).
6. Project data: $Z = X \times W$, where W is the matrix of top k eigenvectors.

Covariance matrix: $C = (1/n) X^T X$ (after mean-centering X)

Eigendecomposition: $C = V \Lambda V^T$

Projection: $Z = X V_k$ (V_k = top k eigenvectors)

Explained variance: $\text{ratio}_i = \lambda_i / \sum \lambda_j$

Full Python Implementation

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

# Load data
```

```

iris = load_iris()
X = iris.data          # 150 samples × 4 features
y = iris.target

# Step 1: Standardize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 2: Covariance matrix
C = (1 / len(X_scaled)) * X_scaled.T @ X_scaled

# Step 3 & 4: Eigendecomposition
eigenvalues, eigenvectors = np.linalg.eigh(C)
# Sort descending
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]

# Step 5: Explained variance
total_var = eigenvalues.sum()
explained_ratio = eigenvalues / total_var
print('Explained variance ratio:', np.round(explained_ratio, 3))
# e.g. [0.729, 0.228, 0.037, 0.006]

# Step 6: Project to 2D
W = eigenvectors[:, :2] # Top 2 eigenvectors
X_pca = X_scaled @ W
print('PCA shape:', X_pca.shape) # (150, 2)

# Quick verify with sklearn
from sklearn.decomposition import PCA
pca_sk = PCA(n_components=2)
X_pca_sk = pca_sk.fit_transform(X_scaled)
print('Sklearn explained ratio:', pca_sk.explained_variance_ratio_)

```



PCA Applications in ML/AI

Visualization: Reduce 500-dimensional text embeddings to 2D for scatterplot visualization.

Noise reduction: Remove dimensions with very low variance (noise).

Feature compression: Compress 1000 features to 50 principal components before training.

Preprocessing for clustering: k-means works much better in lower dimensions.

Face recognition: 'Eigenfaces' — PCA applied to pixel matrices of face images.



Mini Quiz

Q1: What does a principal component with a high eigenvalue represent?

Answer: It is a direction in feature space along which the data varies the most. High eigenvalue = high explained variance = more important direction.

Q2: If PCA retains 95% of variance with 10 components out of 500, what happened?

Answer: We compressed 500 features into 10 without losing much information. The other 490 dimensions contained mostly noise or redundant information.

Q3: Why must we standardize (mean-center and scale) before PCA?

Answer: Without standardization, features with large numeric ranges (e.g., salary in thousands) dominate the covariance matrix, so PCA captures scale differences rather than genuine patterns.

Chapter 7: Singular Value Decomposition (SVD)

7.1 What is SVD?

SVD is the most powerful and universal matrix factorization in all of linear algebra. Every matrix — not just square ones — can be decomposed into three special matrices. It is the Swiss Army knife of data science.

$$A = U \Sigma V^T$$

A : $m \times n$ (original matrix)

U : $m \times m$ (left singular vectors — orthogonal matrix)

Σ : $m \times n$ (singular values — diagonal matrix, non-negative, descending)

V^T : $n \times n$ (right singular vectors — orthogonal matrix)

How SVD Relates to Eigendecomposition

The singular values of A are the square roots of the eigenvalues of $A^T A$:

$$\sigma_i = \sqrt{\lambda_i(A^T A)}$$

The columns of V are the eigenvectors of $A^T A$.

The columns of U are the eigenvectors of $A A^T$.

SVD works on ANY matrix (even rectangular), while eigendecomposition requires square matrices.

7.2 Low-Rank Approximation — Compression with SVD

One of the most useful properties of SVD is that we can approximate any matrix using only the top k singular values and vectors. This gives us the best possible rank- k approximation.

$$A \approx A_k = \sum_{i=1}^k \sigma_i u_i v_i^T \quad (\text{outer product sum of top } k \text{ components})$$

Real-World Analogy

Imagine describing a symphony with just a few dominant themes (the top singular values are like the main themes). You lose some detail, but you capture the essence. With more themes, you get closer to the original. With all themes, you reconstruct the full symphony exactly.

Python: SVD and Image Compression

```
import numpy as np

# Create a sample matrix
np.random.seed(42)
A = np.random.randn(50, 30)

# Full SVD
U, sigma, Vt = np.linalg.svd(A, full_matrices=False)
print('U:', U.shape)          # (50, 30)
print('sigma:', sigma.shape)   # (30,)    - singular values
print('Vt:', Vt.shape)        # (30, 30)

# Reconstruct exactly
A_recon = U @ np.diag(sigma) @ Vt
print('Max error:', np.max(np.abs(A - A_recon))) # ~1e-13

# Low-rank approximation: keep only top k=5 components
k = 5
A_approx = U[:, :k] @ np.diag(sigma[:k]) @ Vt[:k, :]
error = np.linalg.norm(A - A_approx, 'fro') / np.linalg.norm(A, 'fro')
print(f'Relative error (k={k}): {error:.4f}')

# What fraction of 'energy' do k=5 singular values capture?
energy = (sigma[:k]**2).sum() / (sigma**2).sum()
print(f'Energy captured: {energy:.2%}')
```



SVD in Machine Learning Applications

Recommendation Systems (Netflix, Amazon): User-item rating matrix is decomposed via SVD (or matrix factorization). Latent factors represent hidden patterns like 'user likes action movies'.

Natural Language Processing — LSA: Document-term matrix is decomposed by SVD to find latent semantic topics.

Image Compression: An image matrix is decomposed by SVD; keeping only top-k singular values compresses the image.

Pseudo-inverse (A^+): Used to solve ill-conditioned systems: $A^+ = V \Sigma^+ U^T$

Noise Reduction: Small singular values correspond to noise. Zeroing them out removes noise.

SVD vs PCA

Aspect	PCA	SVD
--------	-----	-----

Input	Requires square covariance matrix	Works on any $m \times n$ matrix
Computes	Eigenvectors of covariance	Singular values directly
Numerical stability	Less stable (squares the data)	More stable
Speed	Slower for large data	Faster (especially truncated)
Use case	Dimensionality reduction	General decomposition, compression

Mini Quiz

Q1: If the first 3 singular values of a matrix capture 95% of the total energy, what does that mean?

Answer: The data is essentially 3-dimensional. You can approximate the full matrix with just 3 components and preserve 95% of all its information.

Q2: What matrix does SVD require that eigendecomposition does NOT require?

Answer: SVD works on any $m \times n$ matrix (including rectangular). Eigendecomposition requires a square matrix.

Q3: How is SVD used in recommendation systems?

Answer: The user-item rating matrix is factorized into user embeddings (U), importance weights (Σ), and item embeddings (V). Missing ratings can be predicted from these factors.

Chapter 8: Orthogonality and Projections

8.1 Orthogonal and Orthonormal Vectors

Orthogonal Vectors

Two vectors are orthogonal if their dot product is zero. This means they are at a right angle (90°) to each other — completely 'independent' in direction.

$$u \perp v \Leftrightarrow u \cdot v = 0$$

Example: $u = [1, 0]$ and $v = [0, 1] \rightarrow u \cdot v = 0$ ✓ orthogonal

Orthonormal Vectors

Vectors are orthonormal if they are orthogonal AND each has length 1 (unit vectors). The standard basis vectors $e_1, e_2, \dots, e_n$ are orthonormal.

$$\text{Orthonormal: } u \cdot v = 0 \text{ AND } \|u\| = \|v\| = 1$$

Orthogonal Matrices

A matrix Q is orthogonal if its columns are orthonormal vectors. This gives it a beautiful property: $Q^T = Q^{-1}$. Multiplying by Q preserves lengths and angles.

$$Q^T Q = Q Q^T = I \rightarrow Q^{-1} = Q^T \text{ (very easy to invert!)}$$

Why Orthogonality Matters in ML

Weight initialisation: Orthogonal initialisation in neural networks prevents vanishing/exploding gradients during training.

QR decomposition: Q is orthogonal — makes numerical algorithms fast and stable.

PCA components: Principal components are orthogonal, ensuring they capture non-overlapping variance.

Attention Mechanism: Multi-head attention in Transformers benefits from near-orthogonal query/key subspaces.

8.2 Projections

Projecting a Vector onto Another Vector

The projection of vector b onto vector a gives the component of b that lies along a . It is how much of b points in the direction of a .

$$\text{proj}_a(b) = \frac{(b \cdot a)}{(a \cdot a)} \cdot a$$

Example: $b = [3, 4]$, $a = [1, 0]$

$\text{proj} = ((3 \times 1 + 4 \times 0) / (1 \times 1 + 0 \times 0)) \times [1, 0] = 3 \times [1, 0] = [3, 0]$

This gives the 'shadow' of b onto the x -axis.

Projecting onto a Subspace

To project vector b onto the column space of matrix A (the subspace spanned by A 's columns):

$$\text{Projection matrix: } P = A(A^T A)^{-1} A^T$$

$$\text{Projected vector: } p = Pb$$

Real-World Analogy

Imagine you are standing in a 3D room and shining a flashlight straight down at the floor. The shadow of your hand on the floor is the projection of your hand (3D vector) onto the floor plane (2D subspace). Projection 'flattens' data into a lower-dimensional space.

```
import numpy as np

# Project vector b onto vector a
a = np.array([1., 0.])
b = np.array([3., 4.])

proj = (np.dot(b, a) / np.dot(a, a)) * a
print('Projection:', proj)    # [3. 0.]

# Projection onto subspace spanned by columns of A
A = np.array([[1., 0.],
              [0., 1.],
              [0., 0.]])    # 3D, projecting to xy-plane
b3 = np.array([3., 4., 5.])

P = A @ np.linalg.inv(A.T @ A) @ A.T
print('Projection matrix P:\n', P)
print('Projected b:', P @ b3)    # [3. 4. 0.]
```

```
# Gram-Schmidt orthogonalisation
def gram_schmidt(vectors):
    basis = []
    for v in vectors:
        w = v.copy().astype(float)
        for b in basis:
            w -= np.dot(w, b) * b
        if np.linalg.norm(w) > 1e-10:
            basis.append(w / np.linalg.norm(w))
    return np.array(basis)

vecs = [np.array([1.,1.,0.]), np.array([1.,0.,0.]), np.array([0.,0.,1.])]
orthonormal = gram_schmidt(vecs)
print('Orthonormal basis:\n', np.round(orthonormal, 4))
```

Mini Quiz

Q1: If $Q^T = Q^{-1}$, what is the computational advantage?

Answer: Transposing is free (just read the matrix differently). So finding the inverse costs nothing — just flip rows and columns! This makes orthogonal matrices extremely efficient.

Q2: The projection of $[2, 3]$ onto $[1, 0]$ is $[2, 0]$. What does the residual $[0, 3]$ tell us?

Answer: The residual is the component of $[2,3]$ that is perpendicular to $[1,0]$. It is the part that CANNOT be represented by $[1,0]$ — the information 'lost' by the projection.

Chapter 9: Linear Algebra in Machine Learning

9.1 Linear Regression

Linear regression fits a line (or hyperplane) to data. It is entirely a linear algebra problem.

Model: $\hat{y} = Xw + b$ (matrix-vector product)
 Loss: $L = ||y - Xw||_2^2$ (L2 norm squared)
 Solution: $w^* = (X^T X)^{-1} X^T y$ (Normal Equation)

Key Linear Algebra Concepts Used

- Matrix-vector multiplication: computing predictions Xw
- Transpose: X^T flips the data matrix for gradient computation
- Matrix inverse: $(X^T X)^{-1}$ solves the normal equations
- L2 norm: measures prediction error
- Gradient: $\partial L / \partial w = -2X^T(y - Xw)$

```
import numpy as np

# Generate synthetic data
np.random.seed(0)
n, d = 100, 3
X = np.random.randn(n, d)
w_true = np.array([2., -1., 0.5])
y = X @ w_true + 0.1 * np.random.randn(n)

# Normal equation: w = (X^T X)^{-1} X^T y
w_hat = np.linalg.inv(X.T @ X) @ X.T @ y
print('True weights:', w_true)
print('Estimated: ', np.round(w_hat, 3))

# More stable: use lstsq
w_lstsq, _, _, _ = np.linalg.lstsq(X, y, rcond=None)
print('lstsq:', np.round(w_lstsq, 3))
```

9.2 Neural Networks — Layers as Matrix Multiplications

A deep neural network is literally a sequence of matrix multiplications and non-linear functions. Forward propagation computes the output; backpropagation computes gradients using chain rule and transposes.

Forward pass: $z_i = W_i a_{i-1} + b_i$ $a_i = \text{activation}(z_i)$

Backward pass: $\delta_i = W_i^T \delta_{i+1} \odot \text{activation}'(z_i)$

Weight update: $\Delta W_i = \delta_i a_{i-1}^T$

Key Operations in Neural Networks

- Matrix multiplication: $W @ x$ (linear transformation of activations)
- Transpose: $W.T$ used in backpropagation to route gradients backwards
- Batch processing: process N samples simultaneously with matrix operations
- Jacobian: matrix of partial derivatives used in backprop
- Weight initialisation: often uses orthogonal matrices (He, Xavier init)

```
import numpy as np

def relu(x): return np.maximum(0, x)
def relu_grad(x): return (x > 0).astype(float)

# Simple 2-layer neural network forward pass
np.random.seed(1)
# Input: 4 samples, 3 features
X = np.random.randn(4, 3)

# Layer 1: 3 inputs → 5 hidden
W1 = np.random.randn(5, 3) * 0.1
b1 = np.zeros((5, 1))

# Layer 2: 5 hidden → 2 outputs
W2 = np.random.randn(2, 5) * 0.1
b2 = np.zeros((2, 1))

# Forward pass
Z1 = W1 @ X.T + b1      # (5, 4)
A1 = relu(Z1)           # (5, 4)
Z2 = W2 @ A1 + b2       # (2, 4)
output = Z2              # (2, 4) - 2 outputs per sample
print('Output shape:', output.shape)
```

9.3 Support Vector Machines (SVM)

SVMs find the maximum-margin hyperplane that separates two classes. The entire algorithm is linear algebra: finding a direction w (weight vector) and bias b such that the margin $||w||^{-1}$ is maximised.

Decision boundary: $w^T x + b = 0$

Margin: $2 / ||w||$

Kernel trick: $K(x,z) = \phi(x)^T \phi(z)$ (dot product in feature space)

The Kernel Trick and Dot Products

SVMs with kernels replace dot products with kernel functions. This implicitly maps data to very high dimensions without ever explicitly computing the transformation.

Linear Kernel: $K(x,z) = x^T z$

RBK Kernel: $K(x,z) = \exp(-||x-z||^2 / 2\sigma^2)$

Polynomial: $K(x,z) = (x^T z + c)^d$

9.4 Clustering and k-Means

k-Means clustering assigns data points to clusters by minimising the squared Euclidean distance from each point to its cluster centroid. This is pure linear algebra.

Assignment: $\text{argmin}_k ||x_i - \mu_k||_2^2$

Update: $\mu_k = (1/|C_k|) \sum_{i \in C_k} x_i$ (centroid = mean of cluster)

9.5 Recommendation Systems

Matrix factorization decomposes a user-item rating matrix R (users $\times$ items) into two lower-dimensional matrices: a user embedding matrix U and an item embedding matrix V .

$R \approx U \times V^T$ (matrix factorization)

Predicted rating: $\hat{r}_{ui} = u_u \cdot v_i$ (dot product of embeddings)

```
import numpy as np
from numpy.linalg import svd

# Toy user-item rating matrix (0 = unrated)
R = np.array([[5, 3, 0, 1],
              [4, 0, 4, 1],
              [1, 1, 0, 5],
              [0, 0, 5, 4]])

# SVD factorization
U, sigma, Vt = svd(R, full_matrices=False)
k = 2 # Keep top-2 factors
```

```
# Reconstruct with k=2
R_approx = U[:, :k] @ np.diag(sigma[:k]) @ Vt[:, :k]
print('Original matrix:\n', R)
print('Reconstructed (k=2):\n', np.round(R_approx, 1))

# The approximated values fill in missing ratings!
# R[0,2] was 0 (unrated), prediction: R_approx[0,2]
print('Predicted rating (user 0, item 2):', round(R_approx[0,2], 2))
```

Mini Quiz

Q1: In linear regression, what does the matrix X represent?

Answer: X is the design matrix / feature matrix. Each row is one data sample and each column is one feature. Shape is $(n_{\text{samples}} \times n_{\text{features}})$.

Q2: Why is backpropagation so fast using matrix operations?

Answer: Matrix multiplication is highly parallelisable on GPUs. By representing a full batch of samples as a matrix, all gradients are computed simultaneously with one matrix operation instead of looping over samples.

Q3: What is the 'kernel trick' in SVMs and which linear algebra concept does it exploit?

Answer: The kernel trick computes dot products in a transformed feature space without ever materialising that space. It exploits the fact that inner products (dot products) fully determine geometric relationships.

Chapter 10: Advanced Topics

10.1 The Null Space and Column Space

Column Space

The column space (range) of a matrix A is the set of all vectors that can be written as Ax for some vector x . It is the span of the columns of A . In ML, the column space represents all possible model predictions.

Null Space (Kernel)

The null space of A is all vectors x such that $Ax = 0$. These are the inputs that get 'destroyed' by the transformation — mapped to zero. The null space represents the degrees of freedom that have no effect on the output.

$$\begin{aligned}\text{Column space: } C(A) &= \{Ax : x \in \mathbb{R}^n\} \\ \text{Null space: } N(A) &= \{x : Ax = 0\} \\ \text{Rank-Nullity: } \text{rank}(A) + \dim(N(A)) &= n\end{aligned}$$

10.2 Gradient Descent and Linear Algebra

Gradient descent, the engine behind all deep learning, is a linear algebra algorithm. The gradient is a vector of partial derivatives. Each update step moves parameters in the opposite direction of this gradient vector.

$$\text{Update rule: } w \leftarrow w - \alpha \nabla L(w)$$

Where:

w = weight vector (parameter vector)

α = learning rate (scalar)

∇L = gradient vector (partial derivatives)

```
import numpy as np

# Gradient descent for linear regression
np.random.seed(42)
n, d = 200, 3
X = np.random.randn(n, d)
w_true = np.array([2., -1., 0.5])
y = X @ w_true + 0.05 * np.random.randn(n)

# Initialise
```

```

w = np.zeros(d)
alpha = 0.01 # learning rate

# Gradient descent for 500 steps
for step in range(500):
    y_hat = X @ w # Prediction (matrix-vector product)
    error = y_hat - y # Residual vector
    grad = (2/n) * X.T @ error # Gradient (matrix-vector product!)
    w -= alpha * grad # Update
    if step % 100 == 0:
        loss = np.mean(error**2)
        print(f'Step {step:4d} | Loss: {loss:.6f}')

print('Estimated weights:', np.round(w, 3))
print('True weights:      ', w_true)

```

10.3 Attention Mechanism and Transformers

The Transformer architecture (the backbone of GPT, BERT, and modern LLMs) is built almost entirely on linear algebra, specifically on dot-product attention.

$$\text{Attention} = \frac{QK^T}{\sqrt{d_k}} \text{softmax}(\text{---}) V$$

Q = query matrix, K = key matrix, V = value matrix
 d_k = key dimension (scaling factor)

Attention is Just Scaled Dot-Product Similarity

1. Compute similarity: QK^T (dot products between all query-key pairs)
2. Scale: divide by $\sqrt{d_k}$ (prevents vanishing gradients in softmax)
3. Normalise: apply softmax to get attention weights
4. Aggregate: multiply by values V (weighted sum of value vectors)

This entire mechanism is matrix multiplications and vector operations!

```

import numpy as np

def scaled_dot_product_attention(Q, K, V):
    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k) # Scaled dot products
    # Softmax
    scores_exp = np.exp(scores - scores.max(axis=-1, keepdims=True))

```

```

weights = scores_exp / scores_exp.sum(axis=-1, keepdims=True)
output = weights @ V          # Weighted sum of values
return output, weights

# Example: 4 tokens, d_k=8
np.random.seed(7)
seq_len, d_k = 4, 8
Q = np.random.randn(seq_len, d_k)
K = np.random.randn(seq_len, d_k)
V = np.random.randn(seq_len, d_k)

output, attn = scaled_dot_product_attention(Q, K, V)
print('Output shape:', output.shape)          # (4, 8)
print('Attention weights:\n', np.round(attn, 3))
print('Each row sums to:', np.round(attn.sum(axis=1), 3))

```

10.4 Tensor Operations in Deep Learning

Deep learning frameworks operate entirely on tensors — n-dimensional arrays. Understanding tensor shapes and operations is critical for debugging neural network architectures.

Operation	Description	Example Shape Change
Matrix Multiply (@)	Batch matrix multiplication	$(B,M,N) @ (B,N,P) \rightarrow (B,M,P)$
Element-wise (*)	Hadamard product	$(B,M,N) * (B,M,N) \rightarrow (B,M,N)$
Broadcasting	Operations on different shapes	$(M,N) + (N,) \rightarrow (M,N)$
Reshape	Change layout without copying data	$(B,C,H,W) \rightarrow (B, C*H*W)$
Transpose	Permute dimensions	$(B,H,W,C) \rightarrow (B,C,H,W)$
Sum/Mean	Reduce along axes	$(B,M,N).sum(axis=1) \rightarrow (B,N)$

```

import numpy as np

# Batch matrix multiplication
B, M, N, P = 8, 5, 4, 3
A = np.random.randn(B, M, N)
B_mat = np.random.randn(B, N, P)
C = A @ B_mat    # Shape: (8, 5, 3)
print('Batch matmul:', C.shape)

# Broadcasting
x = np.random.randn(4, 3)    # 4 samples, 3 features
mean = x.mean(axis=0)        # Shape (3,)
x_centered = x - mean         # Broadcasting: (4,3) - (3,) → (4,3)
print('After centering shape:', x_centered.shape)

# Einsum for complex contractions (very useful!)
A = np.random.randn(3, 4)
B = np.random.randn(4, 5)

```

```
C = np.einsum('ij,jk->ik', A, B) # Matrix multiplication
print('Einsum matmul:', C.shape) # (3, 5)
```

Mini Quiz

Q1: If the null space of a transformation has dimension > 0 , what does that mean for the linear system $Ax = b$?

Answer: There are infinitely many solutions (or none). The null space vectors can be added to any particular solution to get another valid solution. The system is underdetermined.

Q2: In gradient descent, why do we use matrix operations instead of loops?

Answer: Matrix operations are vectorised and run in parallel on GPU cores. A single matrix multiply can process thousands of samples simultaneously, making training millions of times faster than Python loops.

Q3: In the attention formula $\text{Attention} = \text{softmax}(QK^T/\sqrt{d_k})V$, what does the dot product QK^T compute?

Answer: It computes the similarity between every query vector and every key vector simultaneously. Large dot product = high similarity = more attention weight.

Chapter 11: Summary and Further Reading

11.1 Core Ideas Recap

Congratulations! You have now covered all the fundamental linear algebra concepts needed for machine learning. Let us consolidate what you have learned.

Concept	One-Line Summary	Key ML Use
Scalar	Single number	Learning rates, loss values
Vector	Ordered list of numbers	Feature vectors, embeddings
Matrix	2D array of numbers	Datasets, weight matrices
Tensor	n-dimensional array	Images, batches, NLP sequences
Dot Product	Sum of element-wise products	Similarity, neuron output
Matrix Multiply	Dot products of rows $\times$ cols	Neural network forward pass
Transpose	Flip rows and columns	Backpropagation, covariance
Inverse	Undo a transformation	Normal equations, solving $Ax=b$
Determinant	Scalar summarising a matrix	Invertibility check
L1/L2 Norms	Measures of vector length	Regularization, distance
Cosine Similarity	Angle between vectors	NLP, document similarity
Linear Independence	No vector is redundant	Feature selection
Eigenvalues/vectors	Special directions of a matrix	PCA, spectral methods
SVD	Universal matrix factorisation	Recommendations, compression
PCA	Find directions of maximum variance	Dimensionality reduction
Projection	Shadow of a vector onto subspace	Least squares, attention
Orthogonality	Perpendicular, dot product = 0	QR decomp, weight init

11.2 How These Concepts Connect

The Big Picture

Linear algebra is not a collection of isolated tricks. It is one unified framework:

DATA lives in high-dimensional vector spaces.

MODELS learn linear transformations (weight matrices) that map input to output.

TRAINING adjusts these transformations using gradient vectors.
EFFICIENCY comes from decomposing matrices into simpler forms (SVD, QR, eigen).
INSIGHT comes from understanding the geometry — distances, angles, subspaces.

11.3 Recommended Next Steps

Mathematics

- Calculus and Multivariable Calculus — for understanding gradients deeply
- Probability and Statistics — for Bayesian methods and probabilistic ML
- Convex Optimisation — for understanding why gradient descent works

Books

- 'Introduction to Linear Algebra' by Gilbert Strang (MIT OpenCourseWare — free videos)
- 'Mathematics for Machine Learning' by Deisenroth, Faisal & Ong (free PDF available)
- 'Deep Learning' by Goodfellow, Bengio & Courville — Chapter 2 covers this material
- 'Linear Algebra Done Right' by Axler — for deeper mathematical foundations

Online Courses

- 3Blue1Brown — 'Essence of Linear Algebra' YouTube series (visual and intuitive)
- MIT 18.06 by Gilbert Strang — complete lecture series on YouTube
- Fast.ai Practical Deep Learning — applies these concepts immediately in PyTorch

Practice Projects

7. Implement PCA from scratch (without sklearn) on the Iris dataset
8. Build a linear regression model using only numpy (normal equations + gradient descent)
9. Implement a simple neural network forward and backward pass with matrix ops only
10. Implement SVD-based image compression on a grayscale photo
11. Implement a simple recommendation system using matrix factorization
12. Visualise word embeddings using PCA/t-SNE on GloVe or Word2Vec vectors

11.4 Final Mini Quiz — Everything



Q1: A dataset has 500 samples and 20 features. What is the shape of the design matrix X ?

Answer: (500, 20) — 500 rows (samples) and 20 columns (features)

Q2: You compute the eigenvalues of a covariance matrix: [8.2, 3.1, 0.5, 0.1]. How much variance is explained by the first 2 principal components?

Answer: $(8.2 + 3.1) / (8.2 + 3.1 + 0.5 + 0.1) = 11.3/11.9 \approx 94.96\%$

Q3: A singular matrix appears in your regression. What does this mean practically?

Answer: Your features are linearly dependent (multicollinearity). The normal equation has no unique solution. You should remove redundant features or add regularization (Ridge regression).

Q4: Why is SVD preferred over eigendecomposition for PCA in practice?

Answer: SVD is numerically more stable (doesn't square the data like computing $X^T X$ does), works on rectangular matrices directly, and modern truncated SVD algorithms are very fast for large sparse datasets.

Q5: Describe in one sentence what matrix multiplication does geometrically.

Answer: It applies a linear transformation to a set of vectors — rotating, scaling, and/or shearing the space — simultaneously for all vectors in the right-hand matrix.

End of Linear Algebra for Machine Learning

Keep practising. Keep building. The maths will become second nature.